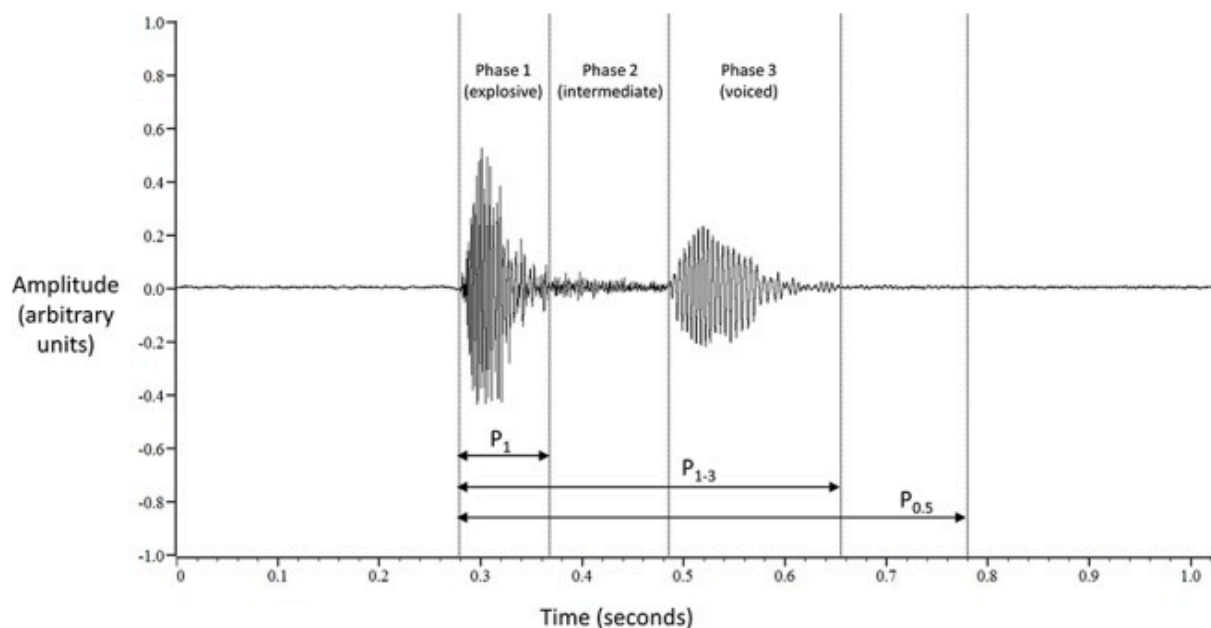


Examen 09/06/25

Práctica

La tos es uno de los síntomas más comunes por el cual se busca consejo médico. Se trata de un mecanismo natural de defensa respiratoria para proteger el tracto respiratorio y un indicador de enfermedad pulmonar. Puede ser caracterizado por una contracción inicial de la musculatura espiratoria contra la glotis cerrada, seguido de una violenta expiración cuando la glotis se abre repentinamente, produciendo un sonido característico. El sonido de la tos generalmente se divide en tres fases: una fase explosiva, un período intermedio, cuyas características son similares a una espiración forzada, y una fase sonora [1].



El objetivo principal de la aplicación a diseñar es obtener un método para el reconocimiento automático y conteo de eventos de tos a partir de grabaciones de sonido, idealmente eliminando la necesidad del entrenamiento de oyentes y permitir un diagnóstico temprano utilizando herramientas de telemedicina.

Como primer paso se desea implementar una etapa de filtrado para extraer los sonidos característicos de la tos asociados a patologías como la bronquitis, ubicados entre los 500 y 1200 Hz [2].

Se cuenta con las señales [tos1.wav](#), [tos2.wav](#), [tos3.wav](#) y [tos4.wav](#) [3], las cuales contienen registros de audio de tos muestreados a 48 kHz con un CAD de 16 bits (V_{ref} : 3,3 V), para un análisis inicial. [*]

Para el diseño del sistema se pide:

1. Determinar los requisitos de diseño de un filtro antialiasing adecuado para la aplicación, teniendo en cuenta que la adquisición de datos se realizará a 8 kHz con 16 bits de resolución (Vref: 3,3 V). El filtro debe poseer una respuesta máximamente plana en la banda de paso.
2. En base a los requerimientos anteriores, diseñe el filtro antialiasing utilizando la herramienta [Analog Filter Wizard](#). Para la implementación utilice una configuración de Múltiples Realimentaciones, resistencias del 10% de precisión y capacitores del 20% (ver [tabla](#)).
3. Utilizando la herramienta pyFDA diseñe un filtro de tipo IIR de orden 8 que permita rescatar el rango de frecuencias de interés. Pruebe el filtro diseñado sobre una de las señales de prueba (remuestreado previamente a 8 kHz). Grafique la señal y su espectro antes y después del filtrado.
4. Para poder extraer parámetros temporales de la señal, es necesario primero calcular la envolvente de la misma. Para esto se debe primero rectificar la señal (`no.abs()`) y luego aplicarle un filtro pasabajos, que en este caso debe tener frecuencia de corte en 20 Hz y atenuar al menos 40 dB a las componentes a partir de 100 Hz. Diseñe un filtro FIR en pyFDA que cumpla con estos requisitos, y realice el cálculo de la envolvente de la señal filtrada en el punto 3. Grafique la envolvente.

* Para cargar las señales puede utilizar los siguientes comandos:

```
from scipy.io import wavfile

archivo_audio = 'tos1.wav'
fs, data = wavfile.read(archivo_audio) # frecuencia de muestreo y datos de la señal

# Definición de parámetro temporales
ts = 1 / fs # tiempo de muestreo
N = len(data) # número de muestras en el archivo de audio
t = np.linspace(0, N * ts, N) # vector de tiempo
senal = data * 3.3 / (2 ** 16 - 1) #se escala la señal a voltios (considerando un CAD de 16bits y Vref 3.3V)
```

Valores normalizados de Resistencias con tolerancia del 10%

1.00	1.20	1.50	1.80	2.20	2.70	3.30	3.90	4.70	5.60	6.80	8.20
------	------	------	------	------	------	------	------	------	------	------	------

Valores normalizados de Capacitores con tolerancia del 20%

1.00	1.50	2.20	3.30	4.70	6.80	-	-	-	-	-	-
------	------	------	------	------	------	---	---	---	---	---	---

Tabla 1 - Valores normalizados de resistencias y Capacitores

[1] Detection of Explosive Cough Events in Audio Recordings by Internal Sound Analysis

[2] Analysis of the Cough Sound: an Overview

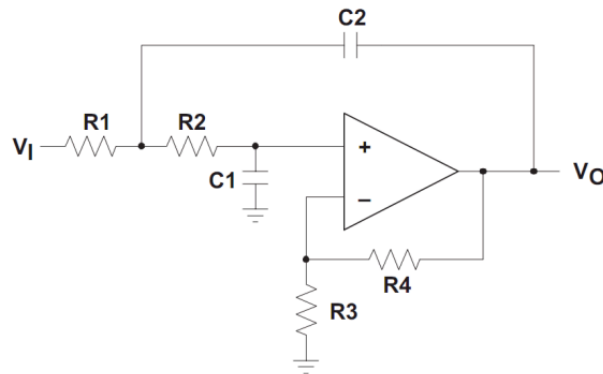
[3] Covid-19 Cough Audio Classification

Teoría

Responda verdadero o falso según corresponda. Justifique su respuesta.

Responder en un archivo de texto a parte

1. El circuito de la figura corresponde a un filtro de Sallen-Key. En el mismo se puede variar la ubicación y número de polos modificando la relación entre R4 y R3.



2. La respuesta al impulso de un filtro FIR es siempre de duración menor o igual al orden del mismo.
3. El algoritmo de filtrado por bloques continuos solapados procesa múltiples ventanas de una señal de la siguiente manera:
 - Realiza la convolución entre un bloque y la respuesta al impulso del filtro
 - Suma los términos transitorio de salida del procesamiento del bloque anterior a los términos del transitorio de entrada del procesamiento del bloque actual.

ENTREGABLES:

Se debe subir un archivo comprimido que contenga:

- Respuestas de examen de teoría:
 - documento de texto: "Teoria_Apellido.txt".
- Resolución de examen de práctica:
 - script de python: "Examen_Apellido.py" (en caso de resolverlo en multiples scripts: "Examen_Apellido_p1.py", "Examen_Apellido_p2.py", etc.).
 - archivos exportados de Analog Filter Wizard (incluyendo simulaciones de LTSpice: ".asc").
 - archivos de filtros digitales (".npz").
 - cualquier otro script con funciones auxiliares u otros archivos necesarios para poder ejecutar correctamente los scripts del examen.

